Assignment 2: Policy Gradient Methods for Predator-Prey Game

EE675: Introduction to Reinforcement Learning

Rudransh Verma (230881)
April 29, 2026

1 Introduction

In Assignment 1, the predator-prey game for an $N \times N$ grid was solved using exact tabular methods (Value Iteration and Policy Iteration). For Assignment 2, we transition to Deep Reinforcement Learning by approximating the policy function using a Neural Network. This document outlines the architectural choices for the policy network, the mathematical implementation of the Monte Carlo policy gradient estimator, and the stochastic gradient ascent training process.

2 Neural Network Design Choices (Part a)

2.1 State Representation and Input Layer

The state space for $N = 4$ contains $|\mathcal{S}| = N^4 = 256$ discrete states. Instead of using a sparse one-hot encoded vector of size 256, the state is represented by a dense continuous vector of size 4, containing the spatial coordinates of the predator and prey: $\mathbf{s} = [p_x, p_y, q_x, q_y]$.

- **Input Dimension ($D_{in} = 4$):** This forces the neural network to learn spatial representations and relative distances, enabling better generalization.
- **Input Normalization:** Neural networks process data most efficiently when inputs are scaled. The raw coordinates $c \in \{1, 2, 3, 4\}$ are Min-Max normalized to the range $[0, 1]$ using the transformation $c' = \frac{c-1}{N-1}$. This ensures stable gradients and prevents spatial coordinates from disproportionately weighting the initial network activations.

2.2 Network Architecture (Hidden Layers)

Because the inputs are scalar coordinate values rather than a high-dimensional image grid, a Multi-Layer Perceptron (MLP) is used instead of a Convolutional Neural Network (CNN).

- **Layer Structure:** A feed-forward architecture with two hidden layers of sizes 64 and 32 respectively. The sequence is: `Linear(4, 64)` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(64, 32)` $\rightarrow$ `ReLU` $\rightarrow$ `Linear(32, 5)`.
- **Capacity Justification:** For a small state space of 256 states, massive over-parameterization is unnecessary and can lead to slower training. A $(64, 32)$ architecture provides sufficient non-linear capacity to map complex coordinate relationships to optimal action distributions without severe overfitting.

2.3 Action Representation and Output Layer

The output layer represents the predator's action space

$$\mathcal{A} = \{\text{Stay, Up, Down, Left, Right, Up-Left, Up-Right, Down-Left, Down-Right}\}$$

- **Output Dimension ($D_{out} = 9$):** The network outputs exactly 9 real-valued numbers representing action logits.
- **Logits vs. Probabilities:** The forward pass of the network returns raw, unnormalized logits rather than softmax probabilities. Applying PyTorch's `Categorical` distribution directly to logits avoids the underflow errors that often occur when taking the log of small softmax outputs during policy gradient calculations.

3 Unbiased Policy Gradient Estimation (Part b)

3.1 Estimation Process (REINFORCE)

To optimize the neural policy, we use the REINFORCE algorithm to obtain an unbiased Monte Carlo estimate of the policy gradient. The gradient of the expected return $J(\theta)$ is given by the Policy Gradient Theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t=0}^{T} G_t \nabla_\theta \log \pi_\theta(a_t|s_t)\right]$$

Where $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_{k+1}$ is the discounted return from time step t onwards.

The `gradient_estimate` function implements this by executing the following steps:

1. **Trajectory Generation:** The agent interacts with the `simulator` using actions sampled from the current policy π_θ. The environment states, received rewards, and action probabilities are recorded. The episode terminates either upon reaching `max_steps` or successfully catching the prey ($r > 0$).
2. **Return Calculation:** The discounted return G_t is computed iteratively backwards from the end of the trajectory.
3. **Objective Construction:** The surrogate objective is constructed by computing the dot product of the log-probabilities and the calculated returns G_t.

3.2 Score Function Implementation in PyTorch

The score function, defined as the gradient of the log of the policy function $\nabla_\theta \log \pi_\theta(a_t|s_t)$, is implemented utilizing PyTorch's automatic differentiation engine alongside the `torch.distributions` module.

Specifically, the following built-in methods are utilized:

- `Categorical.log_prob(action):` When the network outputs logits, a `Categorical` distribution is instantiated. The `log_prob()` method mathematically calculates $\log \pi_\theta(a_t|s_t)$ while maintaining the tensor's connection to the computational graph.
- `torch.autograd.grad():` Once the surrogate objective $\sum G_t \log \pi_\theta(a_t|s_t)$ is constructed, passing it to `autograd.grad()` triggers backpropagation. This automatically computes the analytical derivative (the score function) with respect to all network parameters θ, yielding an unbiased Monte Carlo gradient estimate.

3.3 Variance Reduction via Batch Averaging

While policy gradient methods often use baseline subtraction to reduce variance, an intra-episode baseline (subtracting the mean return of a single episode from its own states) introduces a critical flaw in this specific sparse-reward task. Because the reward is only granted at the terminal step, early optimal actions that initiate the pursuit would mathematically receive a penalty if their discounted return G_t fell below the episode's average.

To preserve the correct reinforcement signals for early actions, the implementation relies on the standard, unbaselined REINFORCE objective. Variance is instead mitigated by accumulating gradients over a batch of $H = 16$ independent episodic trajectories before performing a parameter update, ensuring training stability without distorting the return signals.

4 Stochastic Gradient Ascent and Training (Part c)

4.1 SGA Update Rule and Learning Rates

The policy parameters θ are trained using Stochastic Gradient Ascent (SGA) via the `simple_SGA` function. The update rule is formulated as:

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla_\theta J(\theta_t)}$$

where α is the learning rate. We evaluated the algorithm across three learning rates: $\alpha \in \{0.005, 0.01, 0.02\}$ over 2000 iterations. Optimization is handled without momentum or weight decay to strictly observe the pure policy gradient dynamics.

4.2 Metric Selection: Discounted Return vs. Total Reward

For the learning curve, the primary evaluation metric is the **Discounted Return at $t = 0$ (G_0)** rather than the raw average total reward.

In the 4×4 environment with a horizon of `max_steps=40`, catching the prey is virtually guaranteed even for a highly suboptimal or random policy. Consequently, plotting the raw total reward results in a flat, saturated curve at 1.0, which fails to illustrate the agent's learning progress.

Conversely, the discounted return incorporates a time penalty ($\gamma = 0.99$). A slow capture (e.g., 20 steps) yields a return of $0.99^{19} \approx 0.826$, whereas a rapid capture (e.g., 4 steps) yields $0.99^3 \approx 0.970$. Therefore, an upward trend in the discounted return learning curve strictly demonstrates that the policy is becoming more efficient and optimizing its pursuit paths.

The generated learning curve highlights the unique challenges of training with vanilla policy gradients:

- **The Variance Problem (Signal-to-Noise):** The lower learning rates ($\alpha = 0.005$ and $\alpha = 0.01$) fail to learn, effectively flatlining. Because Monte Carlo gradient estimates are highly noisy, a small learning rate produces a tiny directional signal that gets completely swallowed by the variance. The policy gets trapped on an exploration plateau.

- **Breaking the Plateau:** The highest learning rate ($\alpha = 0.02$) is the only one that succeeds. It provides a mathematical "kick" strong enough to actually shift the action probabilities, overcoming the noise and initiating a positive feedback loop. It begins learning effectively around iteration 750, eventually reaching a discounted return of approximately 0.88.

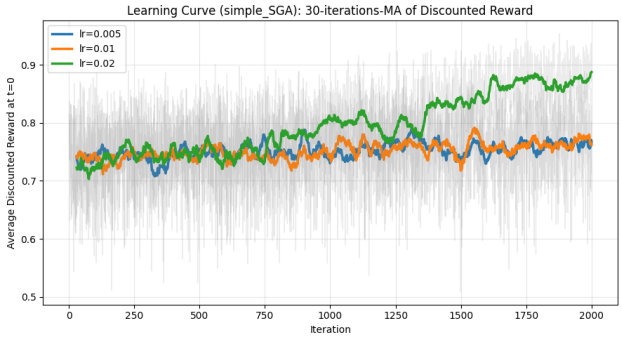


Figure 1: Vanilla SGA learning curve showing the moving average (window=30) of the discounted reward over 2000 iterations.

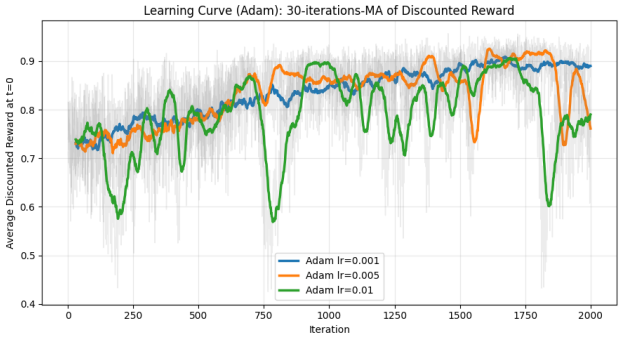


Figure 2: Adam Optimizer learning curve showing the moving average (window=30) of the discounted reward over 2000 iterations.

5 Adam Optimizer and Variance Reduction (Part d)

5.1 Implementation

To address the variance and instability of vanilla SGA, we replaced the optimizer with Adam. Because PyTorch optimizers minimize by default, we perform gradient ascent by negating the gradient estimates before the step: $\theta_{grad} = -\widehat{\nabla_\theta J(\theta_t)}$. We evaluated Adam using lower learning rates: $\alpha \in \{0.001, 0.005, 0.01\}$.

5.2 Learning Curve Analysis (Adam)

The introduction of Adam drastically alters the training dynamics due to its use of momentum and adaptive step sizes, which help filter out the inherent noise of the REINFORCE algorithm:

- **Catastrophic Policy Collapse ($\alpha = 0.01$):** While 0.01 was too small for vanilla SGA, it is much too aggressive for Adam. The curve exhibits massive, sudden drops in reward. A single bad batch of trajectories produces a large, inaccurate gradient step, causing the optimizer to "forget" its good behavior and forcing it to spend hundreds of iterations recovering.
- **Late-Stage Instability ($\alpha = 0.005$):** This rate learns the fastest initially, reaching high rewards by iteration 800. However, as the policy nears an optimal solution, the step size becomes too coarse. It overshoots the optimal weights and bounces out of the stable region, leading to crashes late in training (e.g., near iterations 1550 and 1900).
- **Stable Convergence ($\alpha = 0.001$):** The smallest learning rate proves to be the most reliable. The momentum carries it safely through the noisy gradients without collapsing. It steadily navigates toward a stable, near-optimal policy, hovering tightly around a $0.88-0.90$ discounted return by the end of the 2000 iterations.